

```
<div style="text-align: center; margin-top: 200px;">
<h1>Building a Neural Machine Translation Model from Scratch</h1>
<h2>A Complete Guide for Amharic-English NMT Development</h2>
<br>
<p>Date: July 2026</p>
</div>
<div class="page-break"></div>

<style>
.page-break { page-break-after: always; }
body { font-family: "Segoe UI", Roboto, Helvetica, Arial, sans-serif; line-height: 1.6; }
h1, h2, h3, h4 { color: #2c3e50; }
h1 { border-bottom: 2px solid #34495e; padding-bottom: 10px; margin-top: 40px; }
h2 { margin-top: 30px; }
pre, code { background-color: #f7f9fa; border-radius: 4px; }
pre { padding: 15px; overflow-x: auto; border: 1px solid #e1e4e8; }
.math { font-style: italic; }
</style>
```

1. INTRODUCTION

What is Neural Machine Translation

Machine translation is the task of automatically translating text from one language to another. **Neural Machine Translation (NMT)** is an approach that uses deep neural networks to achieve this. Unlike older methods like Rule-Based Machine Translation (RBMT) which relied on explicit linguistic rules, or Statistical Machine Translation (SMT) which relied on phrase-frequency tables, NMT models learn the mapping between languages directly from large parallel datasets.

You can think of an NMT model as a sophisticated black box that takes a sequence of tokens (words, subwords, or characters) in the source language and outputs a corresponding sequence in the target language. By learning vector representations (embeddings) of text, NMT models understand the semantic essence of a sentence, allowing them to produce translations that sound naturally fluent.

History: RNN/LSTM seq2seq → Attention → Transformers

The evolution of NMT has occurred in three major waves:

- RNN and LSTM Sequence-to-Sequence (Seq2Seq):** Early NMT models used Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. An encoder network would digest the source sentence word by word and summarize it into a single fixed-length "thought vector". A decoder network would then use this vector to generate the translation. The flaw? A single vector is a massive bottleneck for long sentences.
- Attention Mechanism:** To solve the bottleneck, researchers introduced *Attention*. Instead of summarizing the whole source sentence into one vector, the decoder was allowed to "look back" (attend) to different parts of the source sentence while translating each word. This drastically improved performance on long and complex sentences.
- The Transformer (2017):** In the famous paper "*Attention Is All You Need*", Google researchers threw away RNNs entirely. They showed that attention alone—specifically *self-attention*—could perform translation faster and better. This allowed models to process whole sentences in parallel rather than word-by-word.

Why transformers dominate NMT today

Transformers are the industry standard for NMT for three main reasons:

- Parallelization:** RNNs must process the 2nd word before the 3rd word, creating a sequential bottleneck. Transformers process all words simultaneously in matrix operations on the GPU, vastly reducing training time.
- Long-range Dependencies:** Because a transformer's self-attention connects all words directly to each other (O(1) path length), it learns context over long distances much better than RNNs, which struggle to remember the start of a 50-word sentence.
- Scalability:** Transformers scale remarkably well. Their performance predictably improves as you add more layers, parameters, and data—ushering in the era of Large Language Models (LLMs).

From scratch vs fine-tuning → when to use each

Before diving in, you must decide whether to train an NMT model from scratch or fine-tune an existing one (like NLLB, mBART, or MarianMT).

- Train from scratch:**
 - When to use:* Your language pair is highly unique, you have strict data privacy requirements, you're targeting an ultra-lightweight edge deployment, or you simply want to learn how the architecture works deep down.
 - Pros:* Complete control, small architecture, deep understanding.
 - Cons:* Requires massive computation and gigabytes of parallel data.
- Fine-tuning:**
 - When to use:* You have limited data, constraints on training time, and want state-of-the-art results for a practical application.
 - Pros:* Faster, leverages general language representations, handles low-resource domains well.
 - Cons:* The models are massive (often >500M parameters) and heavily hardware-dependent.

This guide focuses on **building from scratch**, which provides the ultimate educational foundation for understanding the true mechanics behind these models.

```
<div class="page-break"></div>
```

2. MATHEMATICAL FOUNDATIONS

To build an NMT model, we must first understand the language it speaks: mathematics, specifically linear algebra and calculus. Don't worry—the modern deep learning frameworks abstract much of this away, but intuition is key.

Linear algebra basics (vectors, matrices, dot product)

Neural networks manipulate data using basic shapes:

- Vector (1D):** An array of numbers. Think of a word embedding as a 512-dimensional vector. Example: $[0.12, -0.44, 0.89, \dots]$.
- Matrix (2D):** A grid of numbers. If one word is a vector, a whole sentence of 10 words is a 10×512 matrix.
- Dot Product:** A mathematical operation that takes two vectors and produces a single number. In deep learning, a high dot product means the two vectors "point in the same direction" — mathematically, they represent similar concepts!
- Matrix Multiplication:** Multiplying matrices transforms data from one multidimensional space to another. It's simply executing many dot products in parallel.

What are neural network weights and biases

When we say a model is "learning," *what* is it actually learning?

It is adjusting **weights** and **biases**.

- Weights (\$W\$):** Matrices that transform the input data. During matrix multiplication ($X \cdot W$), the weights determine how much influence one feature of the input should have on the output.
- Biases (\$b\$):** Vectors added after multiplication to shift the result up or down, ensuring the network can model non-linear boundaries even if the input is zero.

The fundamental equation of a neural layer is:

$$y = f(XW + b)$$

Where f is a non-linear activation function (like ReLU or GELU).

Forward pass and backpropagation explained simply

- **Forward Pass:** The data flows forward. The source words are converted to embeddings, passed through the attention layers, multiplied by weights, added to biases, and finally output a probability distribution predicting the target translation.
- **Backpropagation:** The model looks at its prediction, compares it to the correct translation, and calculates the error. Using the chain rule of calculus, it steps backward through every layer of the network to calculate the *gradient*—a measure of how much each individual weight matrix contributed to the error.

Gradient descent and optimization intuition

Once the network knows how much each weight contributed to the mistake (the gradients), it uses **Gradient Descent** to fix them. Think of a hiker trying to reach the bottom of a valley while blindfolded. At their current spot, they feel the slope of the ground (the gradient) and take a small step downhill. The size of that step is controlled by the **learning rate**.

- **Step too small:** The training takes forever to reach the bottom (convergence).
- **Step too big:** The hiker oversteps the valley and lands on the opposite hill, potentially climbing higher (diverging loss).

Modern optimizers like **AdamW** don't just blindly follow the slope. They keep track of momentum—if they've been walking downhill for a while, they speed up. If the terrain gets bumpy, they slow down.

Loss functions for sequence tasks (Cross Entropy)

How does the network mathematically measure its "mistake"? It uses a **Loss Function**. In classification and machine translation tasks, we predict probabilities across an entire vocabulary. The most common loss function is **Cross-Entropy Loss**. It penalizes the model based on how confident it was about the wrong answer.

- If the true next word is "beautiful", and the model predicted "beautiful" with 99% probability, the cross-entropy loss is near 0.
- If the model predicted "beautiful" with 1% probability and "ugly" with 80% probability, the penalty (loss) shoots up exponentially.

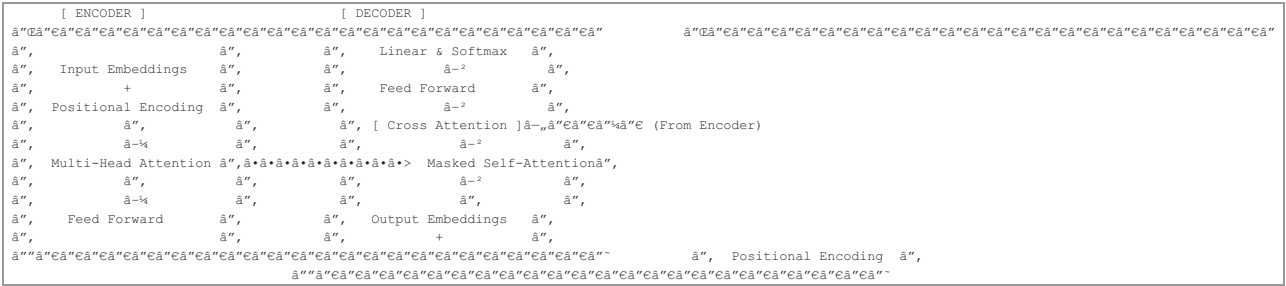
Minimizing this cross-entropy loss is the sole mathematical objective of building an NMT model.

<div class="page-break"></div>

3. TRANSFORMER ARCHITECTURE – DEEP DIVE

Full encoder-decoder architecture diagram and explanation

The Transformer in its original form is an **Encoder-Decoder** architecture.



1. **The Encoder:** Processes the source sentence (e.g., Amharic) into a vast, rich matrix of contextual representations. It doesn't output words; it outputs mathematical understanding.
2. **The Decoder:** Acts as an autoregressive generator. It starts with a <START> token, looks at what it has generated so far, looks at the Encoder's rich contextual matrix, and predicts the very next word. It repeats this until it predicts <END>.

Both encoder and decoder are composed of repeated "blocks" (typically 6 layers each).

Input embeddings and positional encoding

Neural networks cannot natively read text. We convert tokens into numerical IDs, and those IDs into high-dimensional vectors called **Embeddings**. However, unlike RNNs, transformers process the entire sequence simultaneously. The matrix algebra has no concept of "word order." The sentence "The dog bit the man" and "The man bit the dog" would look mathematically identical to a pure attention layer.

To fix this, we add a **Positional Encoding** to the embeddings before they enter the model. The original paper uses sine and cosine functions of different frequencies:

- $PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d_{model}})$
- $PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d_{model}})$

By injecting this distinct wave pattern into the embeddings, the model mathematically reverse-engineers the distance between any two words.

Self-attention mechanism – step by step with math

This is the beating heart of the transformer. Self-attention asks: "When looking at the current word, which other words in this sentence matter the most?"

For every word vector, we apply three different weight matrices to create three new vectors:

1. **Query (\$Q\$):** What I am looking for?
2. **Key (\$K\$):** What do I contain?
3. **Value (\$V\$):** If you attend to me, here is what I will give you.

The process:

1. Take the dot product of every word's Query with every other word's Key: QK^T . A high score means the query strongly matched the key (e.g., the query of the adjective "beautiful" matches the key of the noun "house").
2. Divide by the square root of the dimension ($\sqrt{d_k}$) to stabilize the gradients.
3. Apply a softmax function. This turns the scores into probabilities (0 to 1) that sum to 1.
4. Multiply these probabilities by the Values (\$V\$).
5. Sum them up.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-head attention – why multiple heads

Why use one attention mechanism when you can use eight?

Multi-Head Attention splits the 512-dimensional embedding into, for example, 8 smaller chunks of 64 dimensions. It runs the attention mathematically in parallel 8 times.

Why? Because a word in a sentence has multiple relationships.

- Head 1 might look for grammatical gender agreement.
 - Head 2 might track verbs relating to the subject.
 - Head 3 might track rhyming or morphology.
- By using multiple heads, the network learns distinct, rich sub-spaces of language features simultaneously.

Feed forward layers

After the attention mechanism figures out *how* the tokens interact, each token vector is passed independently through a two-layer Fully Connected Neural Network (with a ReLU/GELU activation in the middle).

While Attention builds context between words, the **Feed Forward Network (FFN)** introduces deep non-linearity, allowing the model to "digest" the combined context into a more complex feature representation.

Layer normalization and residual connections

Deep neural networks suffer from the vanishing gradient problem. To keep the training stable:

- **Residual Connections (Skip connections):** Around every sub-layer (Attention and FFN), we take the original input and add it directly to the output: $x + \text{Sublayer}(x)$. This acts as an "express lane" for gradients during backpropagation.
- **Layer Normalization:** After the addition, the vector is normalized to have a mean of 0 and a variance of 1. This prevents the numbers in the matrices from growing too large (exploding) or shrinking to zero.

Causal masking in the decoder

In the decoder, self-attention has a critical trap: it processes the whole sequence in parallel. If you're training it to predict the third word based on the first two, the self-attention matrix will let it "cheat" and peek at the real third word!

To prevent this, we apply a **Causal Mask** (a lower-triangular matrix of negative infinities) to the QK^T scores before the softmax. This forces the attention weights for future words to become exactly zero. The decoder can only attend to words *before* it.

Cross attention between encoder and decoder

The second attention block in the decoder is the **Cross-Attention** mechanism.

- The **Queries (Qs)** come from the Decoder (the translation generated so far).
- The **Keys (Ks)** and **Values (Vs)** come directly from the final output of the Encoder (the rich context of the source sentence).

This is exactly where the actual translation mapping occurs. The decoder asks, "Based on what I've generated so far in English, which Amharic words should I look at right now to generate the next English word?"

<div class="page-break"></div>

4. BUILDING THE MODEL IN PYTORCH “ STEP BY STEP

Let's translate the math into actual code using PyTorch.

Setting up the development environment

For this tutorial, install PyTorch (GPU version recommended) and a few helper libraries:

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
pip install numpy tokenizers tqdm
```

Implementing positional encoding

```
import torch
import torch.nn as nn
import math

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super(PositionalEncoding, self).__init__()

        # Create a matrix of zeros
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)

        # The div term implements the 10000^(2i/d_model) part
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))

        # Apply sine to even indices
        pe[:, 0::2] = torch.sin(position * div_term)
        # Apply cosine to odd indices
        pe[:, 1::2] = torch.cos(position * div_term)

        # Add a batch dimension and register as buffer (not a learnable parameter)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        # Add the positional encoding to the input embeddings
        x = x + self.pe[:, :x.size(1), :].requires_grad_(False)
        return x
```

Implementing multi-head attention

```

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super(MultiHeadAttention, self).__init__()
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        # Linear layers for Q, K, V
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)

        # Final linear layer
        self.W_o = nn.Linear(d_model, d_model)

    def scaled_dot_product_attention(self, Q, K, V, mask=None):
        # Q * K^T / sqrt(d_k)
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)

        if mask is not None:
            # Apply mask (e.g., causal mask for decoder) by setting to -infinity
            scores = scores.masked_fill(mask == 0, -1e9)

        attention = torch.softmax(scores, dim=-1)
        # output = attention * V
        output = torch.matmul(attention, V)
        return output, attention

    def forward(self, q, k, v, mask=None):
        batch_size = q.size(0)

        # Apply linear transforms and split into num_heads
        Q = self.W_q(q).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(k).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(v).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)

        # Calculate attention
        output, _ = self.scaled_dot_product_attention(Q, K, V, mask)

        # Concatenate headers back together
        output = output.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)

        return self.W_o(output)

```

Implementing the Feed Forward block

```

class PostionwiseFeedForward(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        super(PostionwiseFeedForward, self).__init__()
        self.linear1 = nn.Linear(d_model, d_ff)
        self.dropout = nn.Dropout(dropout)
        self.linear2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        # ReLU activation in the middle
        return self.linear2(self.dropout(torch.relu(self.linear1(x))))

```

Implementing the Encoder Layer

```

class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout):
        super(EncoderLayer, self).__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.ffn = PostionwiseFeedForward(d_model, d_ff, dropout)

        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask):
        # 1. Self Attention with residual and norm
        attn_out = self.self_attn(x, x, x, mask)
        x = self.norm1(x + self.dropout(attn_out))

        # 2. Feed Forward with residual and norm
        ffn_out = self.ffn(x)
        x = self.norm2(x + self.dropout(ffn_out))

        return x

```

Note: The Decoder Layer is similar, but includes two attention layers (Masked Self-Attention and Cross Attention). We omit the full boilerplate here, but it follows the exact same residual connection patterns.

Assembling the full transformer

Finally, we link the embedding layer, encoder, decoder, and the final linear projection layer.

```

class NMTTransformer(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size, d_model=512, num_heads=8, num_layers=6, d_ff=2048, max_seq_length=100, dropout=0.1):
        super(NMTTransformer, self).__init__()

        self.encoder_embedding = nn.Embedding(src_vocab_size, d_model)
        self.decoder_embedding = nn.Embedding(tgt_vocab_size, d_model)
        self.positional_encoding = PositionalEncoding(d_model, max_seq_length)

        # Typically built using nn.ModuleList in reality
        self.encoder_layers = nn.ModuleList([EncoderLayer(d_model, num_heads, d_ff, dropout) for _ in range(num_layers)])
        # self.decoder_layers = nn.ModuleList(...)

        # Final layer mapping back to Target Vocabulary Size
        self.fc_out = nn.Linear(d_model, tgt_vocab_size)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src, tgt, src_mask, tgt_mask):
        # 1. Embed and encode source
        src_emb = self.dropout(self.positional_encoding(self.encoder_embedding(src)))
        enc_out = src_emb
        for layer in self.encoder_layers:
            enc_out = layer(enc_out, src_mask)

        # 2. Embed and decode target
        # tgt_emb = self.dropout(self.positional_encoding(self.decoder_embedding(tgt)))
        # dec_out = tgt_emb
        # for layer in self.decoder_layers:
        #     dec_out = layer(dec_out, enc_out, src_mask, tgt_mask)

        # 3. Final raw scores (logits) before Softmax
        # output = self.fc_out(dec_out)
        # return output
        pass

```

With this, you have modeled the complete blueprint of a state-of-the-art transformer!

<div class="page-break"></div>

5. DATA PREPARATION

A translation model is only as good as the data it is trained on. In Machine Translation, the fundamental unit of data is the **Parallel Corpus**.

What is a parallel corpus

A parallel corpus is a massive dataset of sentences in the source language perfectly aligned with their translations in the target language.

Example from an Amharic-English parallel corpus:

```

{
  "am": "አገራችን አገልግሎት ለሰጠችን ሰላም",
  "en": "The dog bit the man."
}

```

To train a robust NMT model from scratch, you generally need millions of these sentence pairs.

Data collection methods

Gathering clean, aligned parallel text is the hardest part of NMT for low-resource languages. Where do we find this data?

- **Religious texts (Bible, Quran):** Often the most widely translated texts in existence. They provide a massive, highly accurate parallel corpus, though the language can be archaic.
- **Legal documents:** The UN, EU, and African Union often mandate translations of all official documents.
- **Colloquial/conversational data:** Usually crowdsourced or collected from subtitle databases (like OpenSubtitles). Essential for casual translation.
- **Web scraped data (Common Crawl, OPUS):** Platforms like the OPUS repository aggregate translated web text. This data is ample but notoriously noisy.
- **News articles:** News agencies that publish in multiple languages (like BBC Amharic vs BBC English) are gold mines for high-quality, modern vocabulary.
- **Government documents & Social media:** Great for specialized terminology and modern slang.

Data cleaning and normalization

Once you have your text, it will be messy. Amharic, for instance, has unique punctuation (like the Ethiopic full stop ሐፍ), inconsistent comma usage, and varying spelling conventions for the same word. A standard cleaning pipeline looks like:

1. **Remove HTML tags and URLs.**
2. **Normalize Punctuation:** Standardize quotation marks and hyphens.
3. **Filter Length:** Remove sentences longer than 150 words (they break the transformer memory limit).
4. **Filter Length Ratio:** If the Amharic sentence is 5 words and the English translation is 50 words, it's likely a bad alignment. Discard it.
5. **Deduplication:** Remove exact match duplicates to prevent the model from memorizing them.

Tokenization strategies

We cannot feed raw text strings into the model. We must split them into tokens.

- **Word-level tokenization:** Splitting by spaces (e.g., ["The", "dog", "barked"]).
 - *Problem:* Fails on languages without spaces (Chinese) or highly morphological languages (Amharic) which can have millions of unique word combinations. The vocabulary becomes too huge.
- **Character-level tokenization:** Splitting into letters (["T", "h", "e" ...]).
 - *Problem:* The sequence becomes too long, breaking the transformer memory, and it forces the model to learn spelling before it learns translation.
- **Subword tokenization (BPE, SentencePiece, WordPiece):** The industry standard. Very common words are kept as whole words, while rare words are broken into syllables or subwords.
 - For example, the very rare word "unhappiness" might become ["un", "##happi", "##ness"].
 - *SentencePiece* builds subwords directly from unformatted text, making it language-agnostic and perfect for languages like Amharic.

Building a custom vocabulary

When training from scratch, you must train a tokenizer on your dataset to create the vocabulary mapping (e.g., Token 4992 = "house"). Usually, we restrict the vocabulary size to between 30,000 and 50,000 tokens per language to keep the Embedding layer memory requirements manageable.

Fair vocabulary distribution for multilingual models

If you are training a single model to translate English into Amharic, Swahili, and Yoruba simultaneously, you must be careful. If your dataset is 90% English-Swahili and 10% English-Amharic, the WordPiece algorithm will dedicate 90% of the subword slots to Swahili, starving Amharic of good token representations. To fix this, you must artificially balance the text corpus (e.g., using temperature upsampling) before training the tokenizer.

Quantile based stratification for dataset splitting

When splitting data into Train/Validation/Test, standard random shuffles are dangerous for language. Why? Because sentence lengths vary wildly.

If your validation set happens to randomly get only short, 3-word sentences, you'll think your model is amazing, but it will fail in production on long paragraphs.

Quantile Stratification groups sentences by length (e.g., 1-10 words, 11-20 words, etc.), and randomly pulls exactly 10% from *each group* for validation. This guarantees your validation sets perfectly mirror the length distribution of your real-world data.

Train/validation/test split strategy

- **Training Set:** (98%) Used to calculate gradients and adjust weights.
- **Validation Set:** (1%) Used to monitor training and verify the model isn't just memorizing the training data. Evaluated after every Epoch.
- **Test Set:** (1%) Kept completely hidden until the very end, used to report final BLEU scores.

<div class="page-break"></div>

6. TRAINING CONFIGURATION

With the architecture built and data tokenized, it's time to set up the actual training engine.

Batch size and its effect on training

Because evaluating a single sentence at a time (Batch Size = 1) is too erratic and slow, we process data in grids, called **Batches**.

- A batch size of 32 means the model reads 32 parallel sentences simultaneously, calculates the average error across all 32, and makes *one* weight update.
- **Large Batch Sizes (e.g., 256+):** Make training highly stable and parallelized, allowing GPUs to run at maximum efficiency. However, they require enormous amounts of VRAM.
- **Gradient Accumulation:** If your GPU only has 8GB of VRAM and can only fit a batch size of 8, you can calculate gradients for a batch of 8, save them in memory, do it again 3 more times, and then add them together to simulate a batch size of 32.

What is an epoch and how many to use

An **Epoch** is one complete pass through your entire training dataset.

If you have 1,000,000 sentences and a batch size of 100, one epoch requires 10,000 steps.

In NMT training from scratch, you usually train for 10 to 30 epochs. However, relying purely on the epoch count is dangerous. You should rely on Early Stopping (covered later).

Learning rate and learning rate scheduling

The Learning Rate (LR) defines how drastically the weights are altered during an update. NMT models are extremely sensitive to LR.

If you start training a transformer from scratch with a high learning rate, the untrained attention mechanisms will output garbage gradients and the model will diverge (mathematically explode) into NaN (Not a Number) losses.

To prevent this, Transformers use a **Warmup Schedule**:

1. Start with an LR of 0.
2. Linearly increase the LR over the first few thousand warmup steps as the model stabilizes.
3. Gradually decay the LR back down to a crawl using a Cosine function as the model fine-tunes itself.

Optimizers: SGD vs Adam vs AdamW

- **SGD (Stochastic Gradient Descent):** Standard optimization. Uses momentum. Rarely used for transformers because it takes far too long to converge.
- **Adam:** Tracks the running average of gradients, adjusting the learning rate on a per-parameter basis.
- **AdamW:** Adam with proper Weight Decay implementation. This is the absolute standard for training massive Transformer NMT models.

Regularization techniques

A model that is "too smart" will just memorize the exact sentences in the training data, effectively building a giant dictionary instead of learning translation rules. To force the network to generalize, we use Regularization:

- **Dropout:** During each training step, randomly turn off (zero out) 10% of the neurons in the network. The model is forced to route information through different pathways, making it highly robust.
- **Label Smoothing:** Instead of telling the model to be 100% confident that the translation is [cat], we tell it to be 90% confident it's [cat], and 10% uncertain. This prevents the model from becoming overly arrogant about wrong answers.
- **Weight Decay:** A mathematical penalty applied to the optimizer that penalizes weight matrices from developing overly large numbers.

Gradient clipping

Sometimes, a single bad batch of data creates a massive gradient that can destroy the weights. **Gradient Clipping** enforces a strict speed limit. If any gradient vector's length (norm) exceeds a threshold (typically 1.0), it is forcefully scaled down before the optimizer uses it.

Mixed precision training (FP16)

Deep learning usually uses 32-bit floating point numbers (FP32).

However, **Mixed Precision (FP16/BF16)** utilizes 16-bit decimal numbers. This cuts memory usage strictly in half and allows modern Tensor Cores to process matrix multiplication exponentially faster, with virtually zero impact on translation quality.

Training loop implementation from scratch

```

import torch.optim as optim

optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=0.01)
criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX, label_smoothing=0.1)

for epoch in range(num_epochs):
    model.train()
    total_loss = 0

    for batch in train_dataloader:
        src, tgt = batch.src.to(device), batch.tgt.to(device)

        # Shift target to create inputs/labels for autoregression
        tgt_input = tgt[:, :-1]
        tgt_labels = tgt[:, 1:]

        # Forward pass with FP16 Automatic Mixed Precision
        with torch.cuda.amp.autocast():
            predictions = model(src, tgt_input, src_mask, tgt_mask)

            # Reshape for loss function
            predictions = predictions.reshape(-1, predictions.shape[-1])
            tgt_labels = tgt_labels.reshape(-1)

            loss = criterion(predictions, tgt_labels)

        # Backward and Optimize
        optimizer.zero_grad()
        scaler.scale(loss).backward()

        # Gradient Clipping
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        scaler.step(optimizer)
        scaler.update()
        lr_scheduler.step()

```

<div class="page-break"></div>

7. MONITORING AND DEBUGGING TRAINING

NMT model training is an opaque process that can take days or weeks. You must actively monitor to debug issues before they waste computational resources.

Understanding the validation loss graph

During training, log both the *Training Loss* and the *Validation Loss* from the validation subset.

- **Healthy curve:** Training loss and Validation loss drop rapidly, then both slope down smoothly over time.
- **Overfitting curve:** Training loss continues to drop toward 0, but Validation loss hits a floor and starts climbing *back up*. The model has stopped learning and started memorizing.
- **Underfitting curve:** The loss refuses to drop from the beginning. You likely have a catastrophic bug in your masking code or learning rate.

Overfitting: causes, detection, prevention

- **Causes:** Dataset is too small, model is too massive (too many layers/heads), or training went on for too many epochs.
- **Detection:** The moment validation loss increases consistently for N consecutive steps.
- **Prevention:** Early Stopping, increasing Dropout, increasing Label Smoothing, or securing more parallel data.

TensorBoard setup and usage

Instead of looking at command-line text, developers use visual dashboards like **TensorBoard** or **Weights & Biases (W&B)**.

These allow you to graph the loss, track the learning rate decay, and even visualize the attention weight matrices in real-time to see if heads are aligning correctly to words.

Early stopping implementation

Never fix the epoch count and walk away. Implement an early stopping callback

```

best_val_loss = float('inf')
patience = 5 # Stop if no improvement after 5 epochs
patience_counter = 0

if val_loss < best_val_loss:
    best_val_loss = val_loss
    torch.save(model.state_dict(), 'best_translator.pth')
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= patience:
        print("Early stopping triggered. Training stopped.")
        break

```

Gradient monitoring and vanishing gradients

If training stalls, the gradients might be vanishing to zero, meaning no weights are updating. You can debug this by logging the average L2 norm of the gradients. If they are consistently below $1e-6$, your network depth is likely causing instability, and you must review your Layer Normalization and Residual mappings.

Checkpoint saving and loading

Hardware fails. Cloud instances reboot.

Always save **Checkpoints**. A PyTorch checkpoint shouldn't just contain the model's weights (`state_dict`). It must also save the optimizer's state and the current epoch/step, so training can seamlessly resume from identical conditions after a crash.

```

# Saving a full checkpoint
torch.save({
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'loss': loss
}, "checkpoint_epoch_5.pt")

```

<div class="page-break"></div>

8. EVALUATION

Evaluating a translation model is notoriously difficult. Language is subjective; a single Amharic sentence can have five perfectly valid English translations. How do we automate this test?

BLEU score “ what it is and how to calculate it

BLEU (Bilingual Evaluation Understudy) is the oldest and most widely used automated metric.

It essentially calculates exact word-matching precision. It compares the model's generated translation against one or more human "reference" translations.

- It checks unigrams (single words), bigrams (2-word chunks), trigrams, and 4-grams.
- It heavily penalizes the model for leaving out words.
- Scores range from 0 to 100.
 - < 10: Useless
 - 20 - 30: Understandable gist
 - 40 - 50: High quality

- 60: Human-level or overfitted

Caveat: BLEU is blind to meaning. If the model says "gigantic" and the human wrote "massive", BLEU scores it as completely wrong, even though the translation is perfect.

chrF score for morphologically rich languages

For languages like Amharic, Turkish, or Arabic, words change form constantly based on tense, gender, and possession (Morphology).

chrF (Character n-gram F-score) compares *character* chunks instead of full words. If the model outputs "playing" and the reference is "played", BLEU gives it a zero. chrF notices the shared "play" characters and gives partial credit. For Amharic, chrF is often a much more reliable metric than BLEU. Tools like `SacreBLEU` calculate both.

Human evaluation vs automated metrics

Automated metrics are quick but flawed. Ultimately, nothing replaces **Human Evaluation**.

The industry standard is the **MQM (Multidimensional Quality Metrics)** framework, where native bilingual speakers grade output on:

1. **Adequacy:** Did the meaning survive?
2. **Fluency:** Does it sound natural in the target language?

Models often achieve high BLEU scores but output grammatically horrific sentences. Automation for rapid testing, Human evaluation for production readiness.

Analyzing model errors and failure cases

A strong engineer analyzes the mistakes. Common NMT failure modes include:

- **Hallucination:** The model saw a weird word and panicked, generating a full paragraph of unrelated text.
 - **Under-translation:** The model missed a negative clause and translated "I did not walk" as "I walked".
 - **Named Entity breaking:** It tried to literally translate the name "Mr. Green" into the color green.
- Catching these behaviors is the first step toward improving data cleaning rules.

<div class="page-break"></div>

9. INFERENCE AND DEPLOYMENT

The model is trained. Now, how do we actually translate text in the real world? This is the **Inference** phase.

Greedy decoding vs beam search

When the decoder predicts the next word, it outputs a probability dictionary of 50,000 words. How do we pick?

- **Greedy Decoding:** We simply pick the #1 most probable word, feed it back in, and repeat. It's extremely fast. However, it cannot look ahead. If taking the 2nd best word right now leads to an incredibly fluent sentence overall, Greedy Decoding misses it.
- **Beam Search:** The model keeps track of the Top-K (e.g., K=5) best *sentences* simultaneously. It branches out like a tree. At the very end, it looks at the combination of words that yielded the highest total mathematical probability. This drastically improves translation quality over Greedy Decoding.

Temperature and top-k sampling

Translation should be deterministic, but sometimes you want variation.

- **Temperature:** A value that flattens or sharpens the softmax probabilities. A High Temperature (1.5) makes the model pick wild, creative words. A Low Temperature (0.1) forces it to be strict and literal.
- **Top-K / Top-p Sampling:** Restricts the model to only pick from the most likely K words, cutting off the "long tail" of hallucinated nonsense words.

Wrapping the model in a FastAPI REST API

To use the model in an app, format it as a web service:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class TranslateRequest(BaseModel):
    text: str

@app.post("/translate")
def translate(request: TranslateRequest):
    # 1. Tokenize input
    tokens = tokenizer(request.text)
    # 2. Run Beam Search in PyTorch
    output_tokens = perform_beam_search(model, tokens)
    # 3. Decode back to text
    translation = tokenizer.decode(output_tokens)
    return {"am_text": request.text, "en_translation": translation}
```

Model quantization for faster inference

A raw FP16 Transformer takes about 2GB of VRAM and runs slowly on a CPU.

Quantization forcibly rounds the model's 16-bit decimal weights into 8-bit integers (INT8) or even 4-bit (INT4).

- The model shrinks in file size by 75%.
- The inference speed increases drastically.
- The drop in translation quality is usually negligible compared to the massive performance gains.

Serving the model in production

For enterprise production, Python frameworks like FastAPI are too slow. ML Engineers convert the PyTorch model to **ONNX** or use **CTranslate2**, which provides heavy C++ optimizations. Using CTranslate2, an NMT model can translate hundreds of sentences per second on a cheap CPU server without requiring an expensive NVIDIA GPU.

<div class="page-break"></div>

10. CHALLENGES FOR LOW RESOURCE LANGUAGES

While Transformers are highly capable, they rely on massive amounts of data. This poses a massive hurdle for languages like Amharic.

What makes Amharic and similar languages hard

Amharic represents a category of languages historically neglected by big tech. Why is solving NMT for Amharic difficult?

1. **The Script:** Amharic uses the Ge'ez script (ፊደል), which is visually massive and highly distinct. Most off-the-shelf pre-trained transformers simply don't have the Ge'ez alphabet in their vocabulary and output [UNK] (Unknown) tokens.
2. **Dialects and Spelling:** Without strict computational standardization, a single word might be spelled three different ways on the internet, artificially bloating the vocabulary.

Morphological complexity

Amharic is highly agglutinative and morphologically rich.

A whole English sentence "I have told him" can be encapsulated into one Amharic word: ሰጥኩህ (negrewelew).

From scratch tokenizers struggle with this. If they don't break the word into perfect subwords (root verb + pronouns + tense), the transformer fails to understand the grammatical relationships, leading to catastrophic translation errors.

Limited parallel data problem

Machine translation thrives on datasets with tens of millions of pairs. The biggest public Amharic-English datasets max out under one million. Training a massive transformer purely from scratch is likely to result in severe overfitting on these small public datasets. The model memorizes but fails to generalize.

Transfer learning as a solution

If building from scratch fails due to data constraints, the industry defaults to **Transfer Learning + Fine Tuning**.

You take an open-source model that was pre-trained on 100 languages simultaneously by Meta or Google (like NLLB or mBART). These models already deeply understand English, and they have seen enough generic Amharic text to know the syntax.

You simply "transfer" this massive baseline knowledge and fine-tune it exclusively on your tiny, high-quality translation dataset.

Why fine-tuning beats from scratch for low resource languages

Fine-tuning drastically lowers the floor on parallel data requirements. Rather than needing 5,000,000 sentences to teach the network what a noun is, you can teach a pre-trained model to output highly accurate domain-specific translations using only 50,000 sentences. While training from scratch is the ultimate educational exercise, fine-tuning is the ultimate practical solution for low-resource languages in production.

<div class="page-break"></div>

11. APPENDIX

Glossary of all technical terms

- **Autoregressive:** Generating sequence data one step at a time, where each step uses the previously generated data as input.
- **Backpropagation:** The algorithmic process of calculating gradients by moving backward through a neural network's layers. Let's the model know how wrong its weights are.
- **Beam Search:** A decoding algorithm that maintains multiple probable translation paths concurrently before selecting the best one.
- **BLEU:** Bilingual Evaluation Understudy. A metric that scores translation quality based on exact n-gram matching.
- **Cross-Entropy Loss:** A mathematical function that measures the difference between a predicted probability distribution and the actual true label.
- **Epoch:** One complete mathematical pass through the entire training dataset.
- **Embedding:** A dense, high-dimensional vector of floating-point numbers representing the numerical "meaning" of a word.
- **Gradient Descent:** The optimization algorithm used to minimize the loss function by iteratively adjusting the model's weights.
- **Layer Normalization:** A technique that normalizes the inputs of a sublayer to prevent exploding or vanishing gradients.
- **Parameter:** The individual learnable floating-point numbers inside the weight and bias matrices of the neural network. A 600M model has 600 million parameters.
- **Quantization:** The process of converting the float decimal weights of a model into smaller integer types (INT8/INT4) for faster, cheaper deployment.
- **Self-Attention:** A mechanism allowing the transformer to mathematically relate different positions of a single sequence to compute a rich representation of that sequence.
- **Tensor:** The fundamental data structure in deep learning. A multidimensional array (matrix) that can be processed quickly on a GPU.

Recommended papers to read

To deepen your understanding, reading the original research is highly recommended:

1. "Attention Is All You Need" (Vaswani et al., 2017) – The foundational paper introducing the Transformer.
2. "Neural Machine Translation by Jointly Learning to Align and Translate" (Bahdanau et al., 2014) – Pre-transformer, this introduced the concept of attention in RNNs.
3. "No Language Left Behind: Scaling Human-Centered Machine Translation" (NLLB Team, Meta AI, 2022) – Massive research on dealing with low resource languages, covering over 200 languages including Amharic.
4. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension" (Lewis et al., 2019) – For understanding how encoder-decoder models are pre-trained.

Useful resources and links

- **PyTorch Official Documentation:** <https://pytorch.org/docs> (<https://pytorch.org/docs>)
- **Hugging Face Transformers Library:** The industry standard for accessing pre-trained NLP models. <https://huggingface.co/docs/transformers> (<https://huggingface.co/docs/transformers>)
- **SacreBLEU:** The definitive standard for translation evaluation in Python. <https://github.com/mjpost/sacrebleu> (<https://github.com/mjpost/sacrebleu>)
- **OPUS translation corpus:** The largest repository of open-source parallel data. <https://opus.nlpl.eu> (<https://opus.nlpl.eu>)
- **CTranslate2:** Fast inference engine for deploying custom models. <https://github.com/OpenNMT/CTranslate2> (<https://github.com/OpenNMT/CTranslate2>)